

FUNDAÇÃO UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC
CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

ANA LUIZA CAPRISTANO DA CRUZ
EMILLY STEPHANNY DA SILVA
GABRIELA PAULI DE OLIVEIRA
GUILHERME HOERNING REINERT

RELATÓRIO DE ESTRUTURA DE DADOS II:
ÁRVORES AVL, RUBRO-NEGRA E B

JOINVILLE

2026

Sumário

1. Objetivos da Tarefa.....	3
2. Descrição da Implementação.....	3
2.1 Main.c.....	3
2.2 Conjunto.c.....	3
2.3 Avl.c.....	4
2.4 Rubro-negra.c.....	5
2.5 Árvore b.c.....	5
3. Resultados.....	6
4. Conclusão.....	8

1. Objetivos da Tarefa

O objetivo deste trabalho consiste em analisar a complexidade algorítmica das operações de adição e remoção de nós (considerando o balanceamento) em árvores AVL, rubro-negra e B. No caso da árvore B, devem ser considerados os parâmetros de ordem da árvore igual a 1, 5 e 10. A análise deve ser realizada considerando a geração de um conjunto de dados (chaves) com tamanho variando entre 1 e 10000. As chaves devem ser geradas reproduzindo um caso médio, isto é, utilizando chaves aleatórias. Para geração das chaves aleatórias, sugere-se o uso da função `rand` em C considerando uma quantidade de amostra de pelo menos 10 conjuntos para validade estatística, calculando a média dos resultados obtidos.

2. Descrição da Implementação

2.1 Main.c

No arquivo **main.c** são definidos os parâmetros de entrada, como o tamanho dos conjuntos de dados e a quantidade de amostras, além da inicialização do gerador de números aleatórios. Inicialmente, é realizada a semente do gerador utilizando a função **`srand(time(NULL))`**, garantindo a variabilidade na geração das chaves utilizadas nos testes. Em seguida, são definidos os tamanhos dos conjuntos de dados a serem avaliados. Para cada tamanho definido, é criado um conjunto de dados (estrutura **Conjunto**), que é responsável por armazenar os resultados das operações realizadas sobre as estruturas de dados. Ao final da execução, os dados coletados são exportados para um arquivo CSV, o qual será utilizado para a construção dos gráficos de análise.

2.2 Conjunto.c

No arquivo **conjunto.c**, a função **`criarConjunto()`** é utilizada para inicializar a estrutura principal do trabalho, alocando memória para as árvores AVL, Rubro-negra e B (de ordens 1, 5 e 10), além das variáveis responsáveis pelo armazenamento das métricas de desempenho.

A execução ocorre na função **`carregarConjunto()`** que implementa o processo de testes. Nela, são geradas chaves aleatórias utilizando a função **`rand()`**, que são armazenadas nos vetores de inserção (**`adicoes`**) e remoção (**`remocoes`**). Em seguida, essas chaves são utilizadas nas operações de inserção e remoção nas estruturas por meio das funções **`adicionarChaveAVL()`**, **`adicionarChaveRN()`**, **`adicionarChaveB()`** e também **`remocaoAVL()`**, **`removerChaveRN()`** e **`removerChaveB()`**.

Durante essas operações são coletadas métricas de desempenho por meio de acumuladores (**`sumAdicoes`** e **`sumRemocoes`**), que armazenam o esforço computacional de cada estrutura. Após a execução das amostras, é calculada a média do esforço computacional de cada estrutura de dados, a partir dos valores obtidos em cada execução.

Além disso, a função **`carregarConjunto()`** mede, por meio da função **`clock()`**, o tempo de execução dos testes referentes a cada tamanho de conjunto. Já o tempo total de execução de todo o experimento é calculado no arquivo **main.c**.

Por fim, os resultados obtidos são exportados por meio da função **exportarConjuntos()** que gera um arquivo no formato CSV.

2.3 Avl.c

A estrutura AVL foi implementada no arquivo **avl.c**, sendo responsável por manter uma árvore binária de busca autobalanceada durante as operações de inserção e remoção.

A criação da estrutura é realizada pela função **criarAVL()** que inicializa a raiz da árvore como nula. Cada nó da árvore é criado pela função **criarNoAVL()**, que define os ponteiros de pai, filho esquerdo e filho direito.

A inserção de elementos é realizada pela função **adicionarChaveAVL()**, que insere recursivamente os valores na árvore por meio da função auxiliar **adicionarNoAVL()**.

Durante esse processo, é realizada a contagem de operações através do parâmetro **count**, representando o esforço computacional da busca de posição adequada para inserção.

Após a inserção, é chamada a função **balanceamentoAVL()**, responsável por garantir a propriedade de balanceamento da árvore.

O balanceamento é realizado com base no fator de balanceamento calculado pela função **fb()**, que utiliza a altura das subárvores obtidas pela função **altura()**.

Quando o fator de balanceamento ultrapassa os limites $[-1,1]$, são aplicadas as rotações: rotação simples à direita (**rsd()**), rotação simples à esquerda (**rse()**), rotação dupla à direita (**rdd()**), rotação dupla à esquerda (**rde()**).

A remoção é implementada na função **remocaoAVL()**, contendo os três casos: nó folha, nó com um filho, nó com dois filhos. No caso de dois filhos, é utilizado o sucessor em ordem, obtido pela função **encontrarSucessor()**. Após a remoção, o balanceamento é novamente aplicado a partir do ponto afetado.

2.4 Rubro-negra.c

O arquivo **rubro-negra.c** é responsável pelas operações de criação, inserção, remoção, rotações e balanceamento da estrutura. A função **criarRN()** realiza a criação da árvore, inicializando a raiz com um nó sentinela (nulo), utilizado para representar todos os filhos inexistentes. Esse nó simplifica as operações de inserção, remoção e balanceamento, evitando verificações constantes de ponteiros nulos.

Os novos nós são criados pela função **criarNodoRN()**, que recebe como parâmetros o nó pai, a chave a ser armazenada e a cor inicial do novo nó. Conforme as propriedades das árvores rubro-negras, os novos elementos são inicialmente inseridos com a cor vermelha.

A inserção de elementos é realizada pela função **adicionarChaveRN()**, que localiza a posição adequada utilizando a função auxiliar **adicionarNodoRN()**. Após a inserção, a função **balancearRN()** é chamada para restaurar as propriedades da árvore rubro-negra, realizando recolorações e rotações quando necessário.

As rotações utilizadas durante o balanceamento são implementadas pelas funções **rotEsquerdaRN()** e **rotDireitaRN()**, responsáveis por reorganizar os ponteiros entre os nós sem alterar a propriedade de busca da árvore.

A remoção é implementada na função **removerChaveRN()**, que contempla os três casos: remoção de um nó folha, de um nó com apenas um filho e de um nó com dois filhos. Nesse último caso, é utilizado o sucessor em ordem para substituir o elemento removido.

Após determinadas remoções pode ocorrer a violação das propriedades da árvore rubro-negra. Nesses casos, a função **fixBB()** é responsável por restaurar o balanceamento da estrutura nos casos de nós *double-black* por meio de recolorações e rotações, garantindo novamente todas as propriedades da árvore.

Durante as operações de inserção, busca, remoção e balanceamento, o parâmetro **count** é incrementado para contabilizar o esforço computacional realizado pela estrutura.

2.5 Árvore b.c

A implementação da árvore B está no arquivo **b.c**, responsável pelas operações de criação da estrutura, inserção, remoção, divisão de nós, correção de underflow e busca. Diferente das árvores AVL e rubro-negra, cada nó da árvore B pode armazenar diversas chaves e vários filhos, conforme a ordem da árvore.

A criação da estrutura é realizada pela função **criarArvoreB()**, que recebe como parâmetro a ordem da árvore (1, 5 ou 10) e cria a raiz por meio da função **criarNodoB()**. Esta última é responsável por alocar a memória necessária para armazenar o conjunto de chaves e ponteiros para os nós filhos.

A busca pela posição correta de uma chave em cada nó é realizada pela função **pesquisaBinariaB()**, que utiliza pesquisa binária para localizar rapidamente a posição onde a chave está ou deve ser inserida. Essa função é utilizada por praticamente todas as operações da árvore.

A inserção inicia pela função **adicionarChaveB()**, que localiza o nó folha adequado utilizando **localizarNodoB()**. Em seguida, a função **adicionarChaveRecursivoB()** realiza a inserção da chave no nó através da função **adicionarChaveNodoB()**. Caso o número máximo de chaves seja excedido, a função **overflowB()** identifica essa situação e a função **dividirNodoB()** realiza a divisão do nó, promovendo a chave central para o nó pai. Se o pai também ultrapassar o limite permitido, o processo continua recursivamente até que a árvore volte a satisfazer suas propriedades. Caso o nó dividido seja a raiz, uma nova raiz é criada automaticamente.

A remoção é iniciada pela função **removerChaveB()**, que chama recursivamente **removerChaveNodoB()**. Quando a chave está localizada em um nó interno, ela é substituída por seu predecessor antes da remoção efetiva. Após a exclusão da chave, caso um nó passe a possuir menos chaves que o mínimo permitido, ocorre uma situação de underflow.

A correção do underflow é realizada pela função **fixUnderflowB()**, que inicialmente tenta redistribuir chaves entre nós irmãos. Quando essa redistribuição não é possível, é realizada a fusão (merge) entre nós adjacentes por meio da função **mergeB()**. Essas operações garantem que a árvore permaneça balanceada após cada remoção.

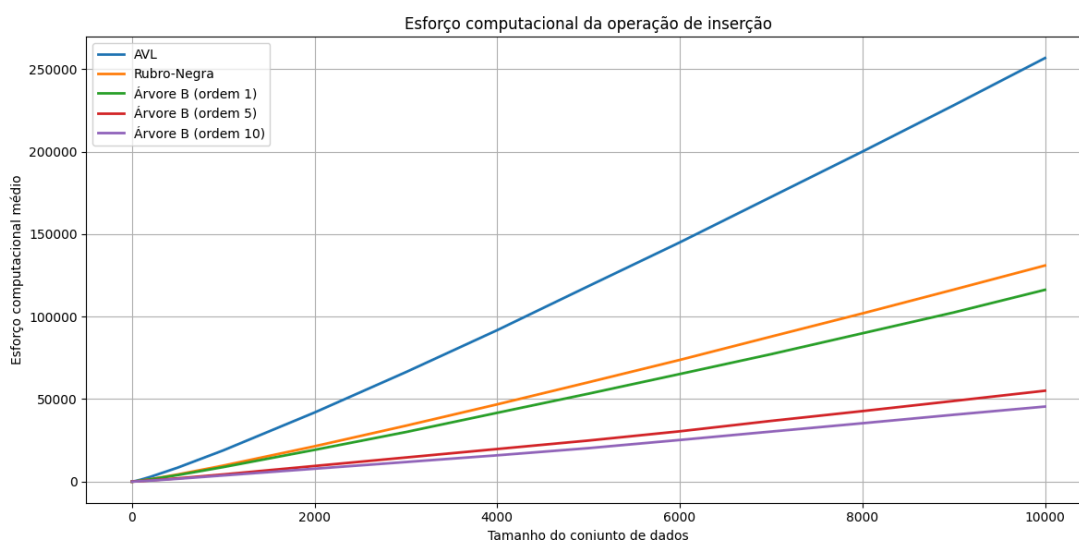
Assim como nas demais estruturas implementadas, o parâmetro **count** é incrementado durante as operações de inserção, remoção e correção da árvore, permitindo contabilizar o esforço computacional.

3. Resultados

Para a realização do experimento, foram avaliados 50 tamanhos de conjuntos de chaves, variando de 200 a 10.000 elementos, em intervalos de 200. Para cada tamanho, foram executadas 10 amostras com chaves aleatórias, sendo calculada a média do esforço computacional obtido nas operações de inserção e remoção. Os resultados foram exportados para o arquivo **conjuntos.csv**, utilizado na geração dos gráficos apresentados a seguir.

O eixo X dos gráficos representa a quantidade de chaves inseridas ou removidas, enquanto o eixo Y representa o esforço computacional médio contabilizado pelas implementações. Foi utilizada escala logarítmica no eixo Y para facilitar a visualização e a comparação entre estruturas cujos valores apresentam diferenças consideráveis. Entretanto, essa escolha altera somente a representação visual dos resultados, não os valores obtidos no processo.

Figura 1 - Esforço computacional médio da operação de inserção

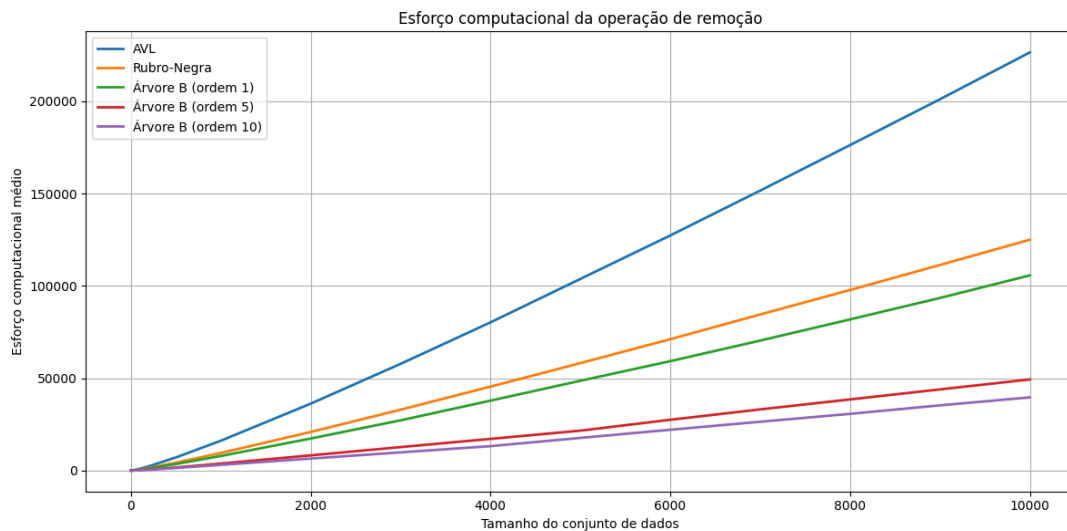


Fonte: Elaborado pelos autores

Conforme apresentado na Figura 1, todas as estruturas demonstraram aumento do esforço computacional à medida que o número de chaves aumentou. Entretanto, foram observadas diferenças claras entre os resultados de cada estrutura: a árvore AVL apresentou o maior esforço computacional médio durante todo o intervalo analisado, enquanto a árvore rubro-negra apresentou valores menores que a AVL, e a árvore B de ordem 1 permaneceu abaixo da rubro-negra na maior parte dos conjuntos avaliados.

As árvores B de ordens 5 e 10 apresentaram os menores valores de esforço computacional. Entre elas, a árvore B de ordem 10 obteve o menor resultado em todos os tamanhos, seguida pela árvore B de ordem 5. Esse comportamento pode ser relacionado à capacidade dos nós de armazenarem um número maior de chaves, reduzindo a altura da estrutura e a quantidade de nós percorridos durante as operações ao mesmo tempo que o esforço de busca por chaves dentro de cada nó aumenta.

Figura 2 - Esforço computacional médio da operação de remoção



Fonte: Elaborado pelos autores

Os resultados da operação de remoção, apresentados na Figura 2, demonstraram um comportamento semelhante ao observado na inserção. Novamente, a árvore AVL apresentou os maiores valores de esforço computacional, seguida pela árvore rubro-negra e pela árvore B de ordem 1.

As árvores B de ordens 5 e 10 mantiveram os menores valores, sendo a árvore B de ordem 10 a que apresentou o menor esforço computacional médio. Embora a remoção possa exigir operações adicionais, como rotações, redistribuições e fusões de nós, a ordem relativa entre as estruturas permaneceu praticamente constante.

A comparação entre os gráficos também demonstra que as operações de inserção e remoção apresentaram tendências semelhantes. Para os dois casos, o esforço computacional aumentou de acordo com o crescimento do conjunto de dados, enquanto as árvores B de maior ordem apresentaram os menores resultados.

4. Conclusão

O trabalho permitiu analisar experimentalmente o comportamento das operações de inserção e remoção nas árvores AVL, rubro-negra e B de ordens 1, 5 e 10. A partir dos resultados obtidos, foi possível observar que o esforço computacional aumentou juntamente com o tamanho dos conjuntos de dados em todas as estruturas avaliadas.

Além disso, segundo o experimento, a árvore AVL apresentou o maior esforço computacional médio nas duas operações. A árvore rubro-negra e a árvore B de ordem 1 apresentaram resultados intermediários, enquanto as árvores B de ordens 5 e 10 demonstraram os menores valores. A árvore B de ordem 10 obteve o menor esforço computacional entre todas as estruturas testadas.

Os resultados indicam que o aumento da ordem da árvore B contribuiu para a redução do esforço computacional observado. Isso ocorre porque nós com maior capacidade de armazenamento permitem que a árvore possua menos níveis, diminuindo a quantidade de nós percorridos durante as operações.

Dessa forma, o experimento demonstrou que as características de organização de cada estrutura influenciam diretamente o desempenho das operações. Entre as estruturas avaliadas, as árvores B de ordens mais elevadas apresentaram os melhores resultados para os conjuntos de dados utilizados.